



Détection de perte de précision dans le cadre de l'analyse statique d'un fichier binaire exécutable

Rapport de stage de 1^e année du Magistère Informatique et Télécommunications, ENS Cachan/Bretagne et Université Rennes 1. Stage effectué du 14/05/2012 au 24/08/2012, soutenance prévue le 07/09/2012.

Résumé

Afin de prouver certaines propriétés sur un fichier exécutable binaire, comme la présence d'erreurs dans une fonction ou l'infection par un *malware* par exemple, il est nécessaire en premier lieu de désassembler le fichier pour en obtenir le code assembleur.

Les instructions de saut à destination variable du type « `jmp eax` » en langage assembleur forment un frein au désassemblage de fichiers binaires. En effet, nous devons connaître précisément les valeurs possibles de `eax` sans quoi le désassemblage ne peut continuer.

Dans ce rapport, nous présentons succinctement quelques techniques pour l'analyse de programme ainsi que des notions plus théoriques dont nous avons besoin, puis proposons un modèle de représentation d'un programme sous forme de structures particulières. A partir de celui-ci, nous présentons une technique mélangeant interprétation abstraite et évaluation de code pour détecter les pertes de précision induites par l'analyse statique.

Bastien Padeloup

Lieu du stage : DGA/MI - Laboratoire AN

Encadrant : Colas Le Guernic

Remerciements

Je tiens tout d'abord à remercier M. Colas Le Guernic qui m'a encadré et conseillé tout au long de ce stage de quatre mois environ, ainsi que l'ensemble des équipes AN, EL et PN de la DGA pour l'accueil chaleureux et pour l'aide apportée tout au long du stage.

Je remercie de plus mes enseignants, et particulièrement Mlle Delphine Demange, M. David Cachera et M. Benoît Cadre pour avoir répondu à mes questions au cours du stage et m'avoir conseillé des documents en relation avec mes interrogations. Merci aussi à M. David Pichardie et M. Thomas Jensen de l'équipe CELTIQUE à l'IRISA pour avoir partagé avec moi les supports de cours du module PAS. Enfin, merci à M. Luc Bougé pour le suivi de ce stage.

Ce stage m'a beaucoup appris, tant sur le plan théorique que pratique, et je remercie à nouveau l'intégralité des personnes qui ont contribué à enrichir ma formation.

Table des matières

Introduction	1
1 Notions essentielles	2
1.1 Analyse de programmes	2
1.1.1 Représentation intermédiaire	2
1.1.2 Graphe de flot de contrôle	3
1.2 Interprétation abstraite	3
1.2.1 Equations représentant la dynamique du programme	4
1.2.2 Domaine concret et domaine abstrait	5
1.2.3 Fonctions d'interprétation	6
1.2.4 Convergence du système associé au programme	6
2 Travail effectué	8
2.1 Problématique	8
2.2 Caractérisation du problème	8
2.3 Emplacement dans la chaîne d'analyse	9
2.4 Compter les valeurs pour détecter un problème	10
2.5 Déterminer des structures particulières dans le CFG	10
2.5.1 Structure de boucle	10
2.5.2 Structure d'indirection	11
2.5.3 Séquence de structures	12
2.6 Estimation par échantillonnage	13
Conclusion	15
Références	16
Annexes	18

Introduction

Depuis les débuts de l'informatique, l'augmentation permanente des performances matérielles et la mise en relation des appareils électroniques ont permis le développement d'applications logicielles de plus en plus complexes. En particulier, il est un type de programmes qui n'a pas échappé à la règle : les virus informatiques. Ces vingt dernières années, de nombreuses techniques ont été développées pour échapper à l'attention de l'utilisateur et des antivirus[7, 2]. Force est de constater que les logiciels antivirus du commerce ne suivent pas la cadence[8]. Il est donc primordial de trouver de nouvelles manières de mettre en avant des erreurs dans des programmes pouvant permettre une infection, ou encore de détecter un comportement malveillant, pour pouvoir continuer la lutte antivirale.

Une de ces techniques consiste à déterminer, pour chaque instruction d'un programme, les valeurs possibles pouvant être prises par les différentes variables quelle que soit la trace d'exécution¹ suivie. La comparaison de ces ensembles de valeurs avec d'autres ensembles caractéristiques de virus permet de détecter la présence ou l'absence d'un programme malveillant au sein du programme analysé[12]. Cette approche diffère des techniques classiques, car elle permet la détection d'un virus par une signature comportementale et non syntaxique, faisant ainsi abstraction de nombreuses techniques d'obfuscation.

Toutefois, l'application de techniques comme celle-ci nécessite d'avoir accès à un code assembleur suffisamment complet pour être analysé, ce qui n'est pas toujours possible (code obfusqué, code source non disponible, ...). L'analyse à partir du code source n'est pas toujours possible ou suffisante, le comportement du programme peut avoir été modifié par des optimisations du compilateur ou par une infection[15] (le code exécuté différant donc du code compilé). Il est donc nécessaire d'analyser un code correspondant le plus possible au programme : le code assembleur obtenu par désassemblage du fichier binaire.

Le désassemblage d'un fichier n'est pas trivial, d'une part car l'architecture Intel x86[1] contient quelques centaines d'instructions, mais surtout car celles-ci sont de tailles différentes et potentiellement disjointes au sein d'un fichier binaire (mélange de données et de code), faisant du désassemblage un problème indécidable. Les désassembleurs utilisent donc des heuristiques et suivent autant que possible le flot de contrôle² pour restituer de manière cohérente la structure du programme.

Une des limites du désassemblage est l'instruction `<< jmp eax >>` (plus généralement les sauts à destination variable) car la destination du saut ne peut être connue qu'au moment de l'exécution, et car une évaluation de code ne permet pas une couverture exhaustive des traces d'exécution. Face à ce type d'instructions, les désassembleurs classiques, sans analyse numérique, doivent stopper leur analyse car il existe un trop grand nombre de suites potentielles au flot de contrôle. Le travail effectué au cours du stage a donc été axé sur la détection de perte de précision lors d'une analyse statique, pour permettre de déterminer le plus précisément possible les destinations potentielles de ce type de sauts, afin de permettre à un désassembleur de continuer son exécution.

Ce rapport sera divisé en deux parties. La première fournira un bref résumé des techniques essentielles à la compréhension du travail réalisé et introduira un certain nombre de notions techniques et formelles. La seconde partie présentera le contenu du stage en tant que tel, à savoir le couplage de techniques d'analyse statique et d'émulation de code pour détecter les pertes de précision induites par l'analyse, et ainsi mieux déterminer les valeurs des registres lors de sauts à destination variable.

1. Une trace d'exécution est une suite d'instructions parcourues lors de l'exécution d'un programme. La trace peut être différente d'une exécution à l'autre, notamment à cause des instructions conditionnelles.

2. Le flot de contrôle est l'ordre dans lequel les différentes instructions sont évaluées au sein d'un programme. Il est souvent représenté par un graphe, qui sera présenté plus en détail au sein de ce rapport.

1 Notions essentielles

1.1 Analyse de programmes

1.1.1 Représentation intermédiaire

Un programme est une suite d'instructions évaluées par une machine. Ainsi, la factorielle d'un entier positif peut être calculée par le programme suivant :

<pre>int fact(n) { int x = 1; while (n > 1) { x *= n; n--; } return x; }</pre>	<pre>fact: pop ebx ; n given from the stack mov eax, 1 ; x initialization startLoopFact: mov ecx, ebx ; n > 1 dec ecx ; n > 1 jle endLoopFact ; if (n <= 1) then jump mul ebx ; x *= n dec ebx ; n-- jmp startLoopFact ; loop endLoopFact: ret ; end (result in eax)</pre>
---	--

FIGURE 1 – Factorielle d'un entier positif en langage C et en langage assembleur Intel x86[1]

Les instructions dans un langage de haut niveau³ sont donc décomposées en instructions plus élémentaires lors de la compilation. Celles-ci sont interprétées par le processeur lors de l'exécution d'un programme.

Toutefois, les différentes instructions du langage assembleur ne sont pas les plus élémentaires possibles. Ainsi, l'instruction `<< add eax, 2 >>` n'a pas pour unique effet d'augmenter la valeur du registre `eax` de 2. En effet, elle incrémente aussi le pointeur d'instruction `eip`, met à jour le registre `EFLAGS` en modifiant les bits associés aux *flags*⁴ `AF`, `CF`, `OF`, `SF`, `PF` et `ZF`, et modifie le contenu des registres `AL`, `AH` et `AX` (ainsi qu'éventuellement `RAX` sur une architecture 64 bits). Il y a donc un grand nombre d'effets de bord qu'il est nécessaire d'explicitier pour pouvoir analyser un programme de manière efficace. Pour cela, nous définissons la représentation intermédiaire d'un programme $\mathcal{P}$.

La notion de langage intermédiaire est très variable d'une publication à l'autre en fonction des besoins. Un très bon langage intermédiaire pour notre propos existe[14] mais nous n'avons pas trouvé d'implémentation en dehors d'une application propriétaire de Google. Nous avons donc choisi de définir notre propre langage. D'autres langages intermédiaires sont donnés en [4] et [18] à titre d'exemples.

Mémoire élémentaire

Une mémoire élémentaire est la donnée :

- d'un nom (`ZF`, `AL`, `0xDEADBEEF`, ...).
- d'une taille en bits (1, 8, 32, ...).
- d'une information (entier, ensemble de valeurs, ...).

Action élémentaire

Une action élémentaire est une unique action parmi :

- lecture d'une mémoire élémentaire.
- écriture dans une mémoire élémentaire.
- opération arithmétique ou logique standard (addition, xor, ...).

Instruction élémentaire

Une instruction élémentaire est une chaîne de caractères dont l'interprétation au sein d'un modèle est l'exécution d'une unique action élémentaire. Un ensemble d'instructions élémentaires est appelé langage intermédiaire.

Représentation intermédiaire

La représentation intermédiaire d'un programme $\mathcal{P}$ écrit dans un langage $\mathcal{L}$ est un programme $\mathcal{P}'$ écrit dans un langage intermédiaire $\mathcal{L}'$ tel qu'il existe deux modèles $\mathcal{M}$ et $\mathcal{M}'$ pour lesquels $[\mathcal{P}]_{\mathcal{M}} = [\mathcal{P}]_{\mathcal{M}'}$.

Une représentation intermédiaire $\mathcal{P}'$ d'un programme $\mathcal{P}$ a donc pour but d'explicitier l'intégralité des effets

3. Le langage C peut être considéré comme de haut niveau, en considérant le langage assembleur comme référence principale.

4. Les *flags* sont des indicateurs permettant de juger du résultat d'un calcul. Ils permettent entre autre de détecter un *overflow*, le signe du résultat, ...

de bord des instructions de $\mathcal{P}$. L'intérêt d'une telle représentation par rapport au programme en langage assembleur est multiple : elle rend possible une analyse plus fine du programme, et est surtout indépendante de la plateforme et du langage de $\mathcal{P}$.

La définition d'un langage intermédiaire n'étant pas le point principal de ce rapport, de plus amples informations sont disponibles en annexe A.

1.1.2 Graphe de flot de contrôle

Certaines instructions, telles que $\ll \text{jz @address} \gg$ ou $\ll \text{jmp eax} \gg$, permettent une altération de l'ordre dans lequel sont évaluées les instructions. Ces instructions, appelées sauts ou encore indirections, ont une source et un ensemble de destinations possibles au sein du programme. Par exemple, si l'instruction $\ll \text{jz @address} \gg$ est située à l'adresse @instruction , alors sa source est l'adresse @instruction , et ses destinations possibles sont @address (l'indirection est prise) et $\text{@instruction} + x$, où x est la taille de l'instruction $\ll \text{jz @address} \gg$ (continuation normale).

Ces changements dans le flot d'exécution permettent de construire un graphe représentant les différentes traces d'exécution possibles du programme considéré :

Bloc de base

Un bloc de base b est une suite d'instructions telle que, quelle que soit la trace d'exécution du programme, si une des instructions de b est exécutée, alors toutes le sont. ie : la première instruction d'un bloc de base est une destination d'un ou plusieurs sauts, et la dernière uniquement est un saut.

Graphe de flot de contrôle

Un graphe de flot de contrôle, communément appelé CFG (de l'anglais Control Flow Graph) est un n -uplet (V, E, v_0, V_f) tel que :

- V est un ensemble de sommets, représentant des blocs de base.
- $v_0 \in V$, point d'entrée du programme.
- $V_f \subseteq V$, blocs finaux⁵.
- E est un ensemble d'arcs orientés qui à tout sommet $v \in V$ associent $n_v \in \mathbb{N}$ sommets $v_i \in V$ ($\forall i \leq n_v$), suites immédiates possibles de v dans le programme.

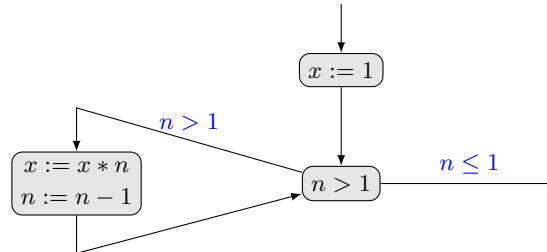


FIGURE 2 – Exemple de graphe de flot de contrôle : $fact(n)$

La construction d'un graphe de flot de contrôle à partir d'un fichier binaire est un domaine de recherche à part entière[11, 19]. Nous ne détaillerons donc pas ici les techniques de désassemblage et de création du CFG.

1.2 Interprétation abstraite

L'interprétation abstraite est une technique d'analyse statique permettant de prouver des propriétés sur un programme à partir d'un ensemble choisi (et réduit au strict minimum nécessaire) d'informations, appelé sémantique collectrice du programme (CS, de l'anglais *collecting semantics*).

Par exemple, soit le programme donné en figure 1. Nous désirons prouver l'assertion suivante : $\forall n, fact(n) > 0$. Une première solution consiste à évaluer la fonction $fact(n)$ pour chacune de ses entrées possibles, et vérifier la positivité de tous les résultats, mais cette méthode n'est clairement pas envisageable étant donné que n peut potentiellement prendre une infinité de valeurs⁶. Une solution est donc de réduire les informations à considérer

5. Les blocs finaux d'un CFG ne sont pas forcément les fins possibles du programme. Ce sont plutôt les limites de ce que nous connaissons du programme (par exemple, ce qui a pu être désassemblé si le programme étudié est issu d'un fichier binaire).

6. Ou un nombre très grand dans le cas d'une implémentation sur ordinateur.

(attention à ne pas confondre avec les valeurs possibles, l'analyse doit rester complète).

Considérons à présent uniquement le signe des variables. Nous savons que le produit de deux nombres positifs donne un nombre positif. Intuitivement, nous voyons bien que nous n'entrerons dans la boucle que si n est strictement supérieur à 1, donc de signe positif⁷. Il en résulte donc que quel que soit le nombre de tours de boucle effectués, x ne sera jamais multiplié que par des nombres positifs. De plus, x est initialisé à une valeur positive et ne change pas de valeur quand l'indirection n'est pas prise. Le résultat sera donc toujours positif, et nous avons pu prouver l'assertion $\forall n, fact(n) > 0$ à partir d'informations plus réduites sur les variables : leurs signes possibles.

Cet exemple intuitif peut être formalisé[3] pour permettre une utilisation plus générique de l'interprétation abstraite. Ainsi, l'analyse d'un programme par interprétation abstraite peut être faite en quatre étapes :

1.2.1 Equations représentant la dynamique du programme

Soit $\mathcal{I}$ l'ensemble des instructions d'un programme. Chaque instruction $i \in \mathcal{I}$ peut être décrite par un certain nombre d'équations (plus précisément $k + 1$, où k est le nombre de destinations possibles de i) qui ont pour but de modéliser la dynamique du programme. On parle alors d'analyse *data flow*[10]. Ces équations sont construites de la manière suivante :

- Soit $in(\mathcal{P})$ l'ensemble des points d'entrée du programme : $\forall i \in in(\mathcal{P}), entry_i = \{(p_0, \top); (p_1, \top); \dots\}$, où $(p, \top)$ indique que la variable p peut prendre n'importe quelle valeur de son ensemble de définition.
- Soit $succ(i)$ l'ensemble des destinations possibles de l'instruction i . Soit $restr(i, j)$ la fonction qui à un sommet i et à une destination $j \in succ(i)$ associe un ensemble de restrictions pour cette destination (par exemple, $\{n > 1\}$). Soit $int(i, c, r)$ l'évaluation de l'instruction i dans le contexte c sous les restrictions r : $\forall i \in \mathcal{I}, \forall j \in succ(i) : exit_{i \rightarrow j} = int(i, entry_i, restr(i, j))$.
- Soit $pred_{exit}(i)$ l'ensemble des résultats des instructions ayant l'instruction i dans leurs destinations possibles, sous la restriction entraînant i comme instruction suivante : $\forall i \in \mathcal{I} : entry_i = \bigcup_{p \in pred_{exit}(i)} (p)$.

Par exemple, le programme donné en figure 1 peut être décrit par le système d'équations suivant :

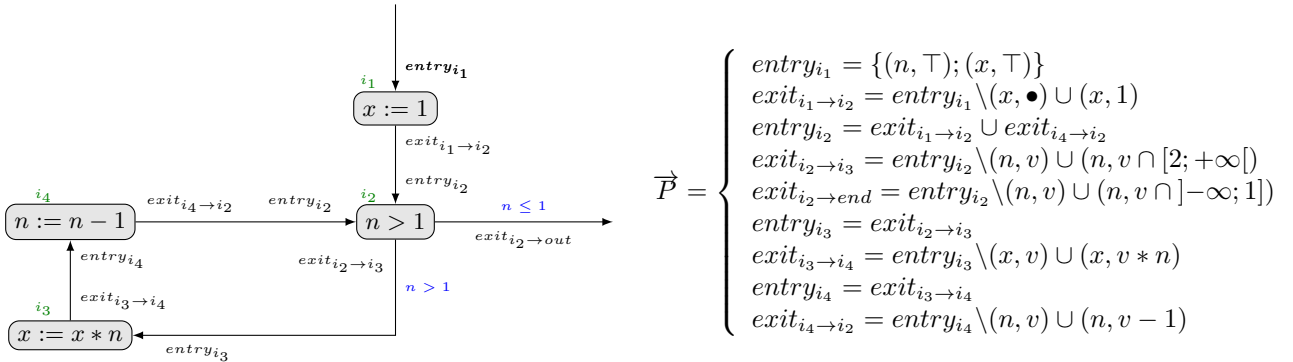


FIGURE 3 – Equations associées au programme $fact(n)$ et leurs *localisations* sur un pseudo-CFG⁸

Dans l'exemple ci-dessus, les variables sont initialisées à $\top$, car elles peuvent contenir n'importe quelle valeur de $\mathbb{Z}$. Les équations du type $exit_{i_1 \rightarrow i_2} = entry_{i_1} \setminus (x, \bullet) \cup (x, 1)$ indiquent que ce qui sort d'une instruction i_1 est l'évaluation de i_1 (affectation de la valeur 1 à x) dans le contexte de ses entrées possibles ($entry_{i_1}$).

De plus, ce qui entre dans un bloc b est l'union des sorties des prédécesseurs directs de b dans le flot de contrôle. Par exemple, la valeur de n en entrée de l'instruction i_2 est l'union de $\top$ et de $[1; +\infty[$. Cette relation est donnée dans $entry_{i_2} = exit_{i_1 \rightarrow i_2} \cup exit_{i_4 \rightarrow i_2}$.

Nous pouvons remarquer que ce système d'équations peut s'écrire sous la forme d'une équation $\vec{X} = F(\vec{X})$, où $\vec{X}$ est un vecteur représentant l'état des variables en entrée et en sortie de toutes les instructions. Cette

⁷. La réalité est plus complexe. En effet, pour des grands nombres, il apparaît des phénomènes d'*overflow* pouvant entraîner que la somme de deux nombres positifs donne un nombre négatif. Nous ferons abstraction de ces difficultés dans cet exemple.

⁸. Ce n'est pas un CFG car chaque nœud représente une instruction et non un bloc de base.

équation est dite de point fixe. Toute solution de ce système contiendra tous les comportements potentiels du programme. Nous nous intéressons à la plus petite solution, contenant moins de comportements, que nous appelons $lfp(F)$ (de l'anglais *least fixed point*).

1.2.2 Domaine concret et domaine abstrait

Comme expliqué précédemment, nous pouvons choisir les informations à considérer dans le programme à analyser (traces d'exécution, valeurs des variables, signes des variables, ...). Ainsi, nous pouvons définir deux domaines pour un programme :

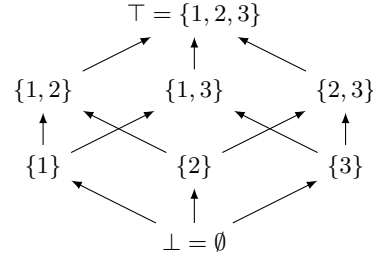
- Le domaine concret : c'est l'ensemble des informations *brutes* collectées lors de l'analyse d'un programme. Il contient les informations jugées utiles pour évaluer la propriété choisie sur le programme analysé, mais contient aussi un certain nombre d'informations à l'utilité superflue. Par exemple, pour répondre à la spécification $\forall n, fact(n) > 0$, nous pouvons considérer les différentes valeurs possibles de n .
- Le domaine abstrait : c'est l'ensemble des informations *raffinées* à partir du domaine concret. Cet ensemble correspond à une plus petite quantité d'informations permettant de répondre à la problématique. Dans notre exemple, l'ensemble des signes possibles pour les variables est un domaine abstrait possible⁹.

Le choix des domaines à considérer n'est toutefois pas quelconque. Pour permettre la convergence du système, les domaines doivent être des treillis complets, reliés par une connexion de Galois[10] :

Treillis complet

Un treillis complet $L = (L, \sqsubseteq) = (L, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ est un ensemble partiellement ordonné $(L, \sqsubseteq)$ dans lequel tout couple d'élément a une plus petite borne supérieure (utilisation de l'opérateur $\sqcup$) et une plus grande borne inférieure (utilisation de l'opérateur $\sqcap$). De plus, $\perp = \sqcup \emptyset = \sqcap L$ est le plus petit élément, et $\top = \sqcup L = \sqcap \emptyset$ est le plus grand élément.

Par exemple, le treillis $L = (\mathcal{P}(\{1, 2, 3\}), \subseteq, \cup, \cap, \emptyset, \{1, 2, 3\})$ est un treillis complet :



Connexion de Galois

(L, α, γ, M) est une connexion de Galois entre les treillis $(L, \sqsubseteq)$ et $(M, \sqsubseteq)$ si et seulement si :

- $\alpha : L \rightarrow M$ et $\gamma : M \rightarrow L$ sont des fonctions monotones.
- $\forall l \in L, \forall m \in M : \alpha(l) < m \Leftrightarrow l < \gamma(m)$.

Dans l'exemple donné par la figure 1, si nous prenons comme domaine concret le treillis $L = (\mathcal{P}(\mathbb{Z}), \subseteq, \cup, \cap, \emptyset, \mathbb{Z})$ et comme domaine abstrait le treillis des signes $L^\#^{10} = (\mathcal{P}(\{-, 0, +\}), \sqsubseteq^\#, \sqcup^\#, \sqcap^\#, \emptyset^\#, \{-, 0, +\})$, nous pouvons définir les fonctions d'abstraction α et de concrétisation γ de la manière suivante :

$$\forall l \in L : \alpha(l) = \begin{cases} \emptyset^\# & \text{si } l = \emptyset \\ \{0\} & \text{si } l = \{0\} \\ \{-\} & \text{si } l < 0 \\ \{+\} & \text{si } l > 0 \\ \{0, -\} & \text{si } l \leq 0 \\ \{0, +\} & \text{si } l \geq 0 \\ \{-, 0, +\} & \text{si } l \text{ quelconque} \end{cases} \quad \forall l^\# \in L^\# : \gamma(l^\#) = \begin{cases} \emptyset & \text{si } l^\# = \emptyset^\# \\ \{0\} & \text{si } l^\# = \{0\} \\]-\infty; 0[& \text{si } l^\# = \{-\} \\]0; +\infty[& \text{si } l^\# = \{+\} \\]-\infty; 0] & \text{si } l^\# = \{0, -\} \\ [0; +\infty[& \text{si } l^\# = \{0, +\} \\]-\infty; +\infty[& \text{si } l^\# = \{-, 0, +\} \end{cases}$$

9. Les domaines ne sont pas uniques. Nous aurions aussi pu utiliser par exemple l'ensemble des intervalles possibles pour les valeurs des variables comme domaine abstrait.

10. Le symbole $\#$ est fréquemment utilisé pour distinguer ce qui appartient à un domaine abstrait.

1.2.3 Fonctions d'interprétation

Maintenant que nous avons choisi un domaine concret et un domaine abstrait, il reste à définir une sémantique pour notre programme permettant de manipuler les données selon la représentation choisie. Ainsi, pour le programme donné par la figure 1 et décrit par le pseudo-CFG en figure 3, nous pouvons généraliser le système de la manière suivante :

$$\vec{P} = \begin{cases} entry_{i_1} = \{(n, \top); (x, \top)\} \\ exit_{i_1 \rightarrow i_2} = int(i_1, entry_{i_1}, \emptyset) \\ entry_{i_2} = exit_{i_1 \rightarrow i_2} \cup exit_{i_4 \rightarrow i_2} \\ exit_{i_2 \rightarrow i_3} = int(i_2, entry_{i_2}, \{n > 1\}) \\ exit_{i_2 \rightarrow end} = int(i_2, entry_{i_2}, \{n \leq 1\}) \\ entry_{i_3} = exit_{i_2 \rightarrow i_3} \\ exit_{i_3 \rightarrow i_4} = int(i_3, entry_{i_3}, \emptyset) \\ entry_{i_4} = exit_{i_3 \rightarrow i_4} \\ exit_{i_4 \rightarrow i_2} = int(i_4, entry_{i_4}, \emptyset) \end{cases}$$

La fonction d'interprétation $int(i, c, r)$ peut alors être changée pour permettre une évaluation concrète différente. Par exemple, si nous utilisons le treillis concret $L = (\mathcal{P}(\mathbb{Z}), \subseteq, \cup, \cap, \emptyset, \mathbb{Z})$, nous avons la fonction suivante :

$$\forall v \in L : int(i, c, r) = \begin{cases} c \setminus (x, \bullet) \cup (x, \{1\}) & \text{si } i = i_1 \\ c \setminus (n, v) \cup (n, v \cap [2; +\infty[) & \text{si } i = i_2 \text{ et } r = \{n > 1\} \\ c \setminus (n, v) \cup (n, v \cap]-\infty; 1]) & \text{si } i = i_2 \text{ et } r = \{n \leq 1\} \\ c \setminus (x, v) \cup (x, v * n) & \text{si } i = i_3 \\ c \setminus (n, v) \cup (n, v - 1) & \text{si } i = i_4 \end{cases}$$

1.2.4 Convergence du système associé au programme

En utilisant l'équation de point fixe $\vec{X} = F(\vec{X})$, nous pouvons à présent définir la fonction suivante :

$$\left| \begin{array}{ll} f : L^\# & \rightarrow L^\# \\ l^\# & \mapsto (\alpha \circ F \circ \gamma)(l^\#) \end{array} \right.$$

L'intérêt d'une telle fonction est sa monotonie (ie : $\forall x, y \in L : x < y \Rightarrow f(x) < f(y)$). En effet, nous disposons du théorème suivant :

Théorème : Convergence d'une fonction monotone sur un treillis complet (Tarski)

Soit L un treillis complet. Toute fonction $f : L \rightarrow L$ monotone admet au moins un point fixe.

Ce théorème assure qu'il existe un point fixe pour f (ie : une solution de l'équation $f(x) = x$). Toutefois, ce point fixe peut être trop *haut* dans le treillis, et donc être trop imprécis pour répondre à la problématique initiale. Nous pouvons résoudre ce problème grâce au théorème suivant :

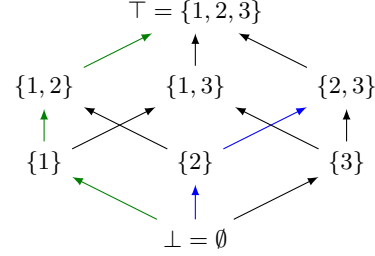
11. Ici, le produit est une fonction $*$: $L \rightarrow L$ dont l'ensemble d'arrivée est l'union des résultats possibles du produit de deux éléments $i \in v$ et $j \in n$.

12. De même, la soustraction est une fonction $-$: $L \rightarrow L$ dont l'ensemble d'arrivée est l'union des résultats possibles de la soustraction de deux éléments $i \in v$ et $j \in n$.

Une chaîne ascendante sur un treillis $L = (L, \sqsubseteq)$ est une suite croissante (au sens de la relation d'ordre partiel $\sqsubseteq$) d'éléments de L . Dans une chaîne ascendante $(l_n)_n$, pour tout n , $l_n \sqsubseteq l_{n+1}$

Par exemple, sur le treillis $L = (\mathcal{P}(\{1, 2, 3\}), \subseteq, \cup, \cap, \emptyset, \{1, 2, 3\})$, les suites $l_1 = (\emptyset, \{1\}, \{1, 2\}, \{1, 2, 3\})$ et $l_2 = (\emptyset, \{2\}, \{2, 3\})$ sont des chaînes ascendantes :

Chaîne ascendante



Théorème : Théorème du point fixe (Kleene)

Soit $f : L \rightarrow L$ une fonction mononone. Alors $lfp(f)$ est la borne supérieure de la suite $\perp \leq f(\perp) \leq f^2(\perp) \leq \dots \leq f^n(\perp) \leq \dots$

Ce théorème nous permet de définir simplement un algorithme pour obtenir le plus petit point fixe de f . En effet, si le treillis est fini, nous avons la relation suivante : $\exists n \in \mathbb{N} : lfp(f) = f^n(\overrightarrow{\emptyset^\sharp})$ ¹³. Un algorithme détaillé est donné en annexe B.

Le théorème n'assure toutefois pas la convergence en un temps fini. Nous n'entrerons pas dans les détails, mais il est possible de définir un opérateur ∇ de *widening*[3] qui permet d'ignorer un très grand nombre d'itérations pour atteindre un post-point fixe de f (ie : un élément f^∇ tel que $f \sqsubseteq f^\nabla$), assurant la convergence en un temps fini tout en conservant les propriétés recherchées sur le programme (mais en induisant une perte de précision supplémentaire).

Une fois le point fixe atteint, il suffit de vérifier les propriétés désirées dans le système $lfp(f)$. En effet, les treillis L et $L^\sharp$ étant reliés par une connexion de Galois, nous avons l'assurance que $lfp(F) \subseteq \gamma(lfp(f))$ [10]. Pour résumer, tout ce qui peut être prouvé dans le domaine abstrait peut aussi l'être dans le domaine concret.

13. Par convention, $\overrightarrow{\emptyset^\sharp}$ est généralement noté $\overrightarrow{\perp}$.

2 Travail effectué

2.1 Problématique

Un fichier binaire n'est qu'une suite de 0 et de 1. Le code et les données sont mélangées à la compilation, et différencier l'un de l'autre est un problème indécidable. Les désassembleurs ne peuvent donc pas simplement désassembler des blocs contigus de bits de taille fixe, mais doivent suivre le flot de contrôle pour reconstruire le programme.

Les indirections à destinations multiples, telles que `<< jmp eax >>` ou `<< jz eax >>` par exemple, posent donc un problème. En effet, si une analyse révèle que le registre contenant l'adresse de destination du saut contient potentiellement n valeurs distinctes, alors il faudra tenter de désassembler du code sur chacune des n branches concernées.

Plus grave que la perte de temps occasionnée par un mauvais branchement, si une mauvaise suite de bits est analysée, elle peut toutefois avoir une signification. Prenons un exemple simple : nous disposons des instructions fictives `<< A = 01101010 >>`, `<< B = 01010011 >>`, `<< C = 11111011 >>` et `<< D = 01111111 >>` ainsi qu'une instruction de saut `<< jmp x >>`, où x est un registre. Soit la portion de programme suivante : 01101010 01111111 01101010. Une analyse préalable nous indique que nous pouvons sauter au premier bit ou au quatrième de cette séquence. Ainsi, en explorant la première voie, nous trouvons que la suite logique du programme est la séquence [A, D, A] alors qu'avec la seconde destination nous trouvons la séquence [B, C] (01101010 01111111 01101010).

Si la valeur indiquant qu'il est possible de sauter au quatrième bit est erronée (incluse par sur-approximation lors de l'analyse), nous n'avons aucun moyen de savoir que le code associé à cette adresse ne peut être atteint réellement. Cela peut résulter en la détection de faux positifs dans le cadre d'une analyse de malware, par exemple.

Il est donc critique de pouvoir déterminer avec précision les destinations potentielles de sauts dynamiques pour pouvoir continuer le désassemblage du programme à analyser.

2.2 Caractérisation du problème

L'analyse statique par interprétation abstraite est une technique qui peut permettre de connaître, pour chaque instruction et pour chaque mémoire élémentaire, une sur-approximation de l'ensemble des valeurs pouvant être prises par ces dernières. Comme nous l'avons vu précédemment, il est nécessaire de perdre de la précision sur la représentation de ces valeurs pour pouvoir représenter des plages de valeurs potentiellement infinies. Cette perte de précision se fait par le choix du treillis sur lequel sont définies les variables. Ainsi, si nous choisissons de représenter les valeurs par des intervalles, comme ce sont des ensembles convexes, il en résulte que de nombreuses valeurs seront incluses par sur-approximation. Un exemple est donné par la figure suivante :

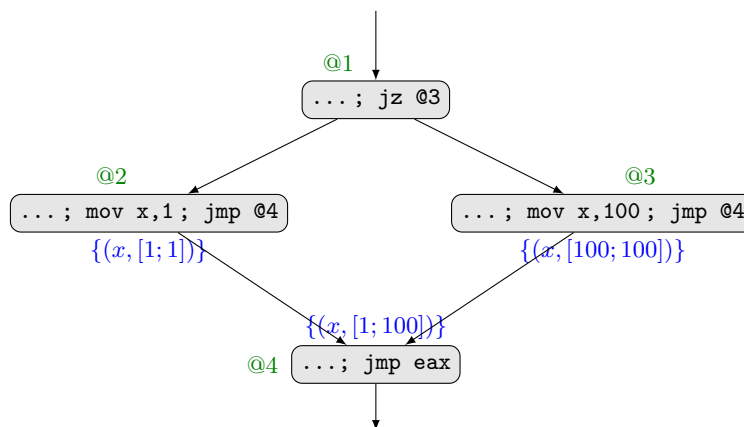


FIGURE 4 – Perte de précision liée à une représentation convexe des ensembles

Dans cet exemple, l'union des valeurs de x en entrée du bloc à l'adresse @4 fait que les valeurs [2; 99] sont incluses par sur-approximation.

Des travaux existent pour limiter la perte de précision, comme par exemple l'utilisation du domaine des *k-sets*[19] consistant à fixer un seuil maximal de valeurs possibles pour les variables, stockées comme un ensemble d'éléments jusqu'à ce que le seuil soit dépassé. Les variables concernées passent alors à $\top$. Un graphe est ensuite généré pour tracer les conséquences des mises à $\top$ des variables de toute instruction i sur toute instruction i' suivante dans le flot de contrôle. Enfin, une augmentation progressive du seuil sur certaines variables permet de limiter les conséquences de la perte de précision dans la suite du programme.

À l'opposé des techniques statiques, il existe des méthodes dites *dynamiques*, dans lesquelles nous allons évaluer le code au sein d'une machine virtuelle. Ces techniques ne peuvent être complètes car il est très difficile d'être exhaustif sur les entrées possibles d'un bloc d'instructions, mais sont extrêmement précises. Par exemple, si un bloc d'instructions b ne manipule qu'une variable x pouvant valoir $[1; 2]$, nous pouvons évaluer le corps de b en affectant les valeurs individuellement à x . Il en résulte que nous obtiendrons les valeurs réelles pouvant être prises par x en sortie de b . Dans le cas où de nombreuses configurations des variables sont possibles en entrée, une évaluation de code pour un échantillon de ces configurations permet l'obtention d'une sous-approximation des valeurs possibles en sortie de b .

L'idée est donc de mélanger ces deux techniques. L'interprétation abstraite aura pour objectif de nous fournir une première idée des valeurs possibles des mémoires, et l'évaluation de code permettra un raffinement quand cela s'avère possible. Ainsi, quand le nombre de n -uplets possibles en entrée d'un bloc b est petit ¹⁴, il est possible d'évaluer b pour l'intégralité de ses configurations d'entrée.

Par exemple, si nous avons un bloc b dont l'exécution ne dépend que d'une variable x tel que $entry_b = \{(x, [0; 2])\}$ et $exit_b = \{(x, \top)\}$ alors, comme l'ensemble de valeurs possibles pour x est fini et petit en entrée, nous allons pouvoir évaluer b pour chaque valeur de x ¹⁵. Nous aurons alors un nouvel ensemble pour x ¹⁶ et nous pourrons recommencer une analyse par interprétation abstraite à partir de ce point pour raffiner la suite du programme, jusqu'à l'instruction de saut qui nous intéresse.

Cette technique est toutefois gourmande en ressources. En effet, si le bloc d'instructions à analyser manipule k variables de n valeurs, alors la complexité de l'évaluation sera de $O(n^k)$ (exponentielle en nombre de variables). Il est donc important de bien choisir dans quelles circonstances appliquer cette technique.

Pour cela, nous allons procéder en deux temps. Une première phase consistera à découper le CFG associé au programme à analyser en zones particulières pour lesquelles nous montrerons certaines propriétés, puis une seconde étape sera l'analyse de ces zones à plusieurs niveaux pour détecter des explosions du nombre de valeurs pour certaines variables.

Cette détection d'explosions sera suivie d'une correction des pertes de précision, soit par une évaluation de code pour tous les n -uplets de valeurs des variables, soit par exemple par un appel à un analyste.

2.3 Emplacement dans la chaîne d'analyse

Etant donné un programme binaire à analyser, une première phase a lieu pour en extraire le CFG. Il faut en premier lieu le désassembler pour obtenir un programme en assembleur type CISC ¹⁷. Ce programme est ensuite converti en assembleur RISC ¹⁸ car certaines instructions en CISC sont de véritables algorithmes (par exemple, `<< loop @address >>`).

Ces instructions sont ensuite transposées dans le langage intermédiaire choisi, puis le CFG est construit par un programme choisi.

Vient ensuite l'analyse réelle du programme. Une première interprétation abstraite fournit les ensembles de valeurs possibles pour les variables. Une recherche de problème est faite pour déterminer un endroit où l'analyse peut être raffinée. Si un tel endroit est trouvé, une résolution de problème est effectuée ¹⁹. Une fois le

14. Peu de configurations possibles en entrée. Dans le cas d'un treillis non-relationnel, ce nombre est le produit des cardinaux des ensembles de valeurs de toutes les variables définies.

15. Il faudra toutefois faire attention à placer une *garde*, c'est-à-dire un compteur ou un *timer* forçant l'arrêt de l'analyse à un certain moment (en temps ou en nombre d'instructions) pour assurer la terminaison en cas de boucle infinie par exemple.

16. Il faudra potentiellement changer de mode de représentation pour les données et ne pas conserver des intervalles. En effet, l'ensemble $\{1; 1000\}$ se représente par $[1; 1000]$. Il aurait été plus intéressant de représenter les valeurs par l'ensemble lui-même.

17. *Complex Instruction Set Computing* : http://en.wikipedia.org/wiki/Complex_instruction_set_computing.

18. *Reduced Instruction Set Computing* : http://en.wikipedia.org/wiki/Reduced_instruction_set_computing.

19. Cela peut être une évaluation de code, mais d'autres techniques peuvent être imaginées, comme simplement l'intervention humaine de l'analyste.

problème corrigé, une nouvelle interprétation abstraite est réalisée en prenant en compte ces valeurs plus précises.

L'analyse s'arrête quand il n'y a plus d'endroits intéressants à raffiner, ou du moins quand ils ne peuvent plus être détectés automatiquement. L'analyste prend alors le relais.

Le nœud du diagramme 8 donné en annexe C apparaissant en rouge indique l'emplacement au sein de la chaîne d'analyse où se situe notre travail.

2.4 Compter les valeurs pour détecter un problème

Pour détecter de la perte de précision causée par la représentation des valeurs des variables par un ensemble convexe lors d'une analyse, nous avons décidé d'analyser le comportement des cardinaux des ensembles.

Proposition : Si une structure s a k configurations possibles de n variables manipulées par s , alors il est possible de majorer le nombre de configurations en sortie de s par k . Notons cette borne $\text{sup}(s)$.

Idée : Soit l'instruction i en langage intermédiaire $x := \text{mul}(y, z)$ effectuant la multiplication des variables y et z et stockant le résultat dans x . S'il existe k configurations possibles des variables y et z , alors en énumérant sur ces couples, nous aurons au maximum k résultats en sortie. Dans le cas où y et z sont des variables indépendantes, alors il existe autant de configurations possibles que le cardinal du produit cartésien de y et z .

De plus, une instruction i étant une suite déterministe d'instructions élémentaires, nous pouvons donc déterminer une borne supérieure pour toute variable manipulée par les instructions élémentaires constituant i . C'est en effet la valeur de $\text{sup}(j)$, où j est la dernière instruction élémentaire de i . Enfin, un bloc de base est une suite déterministe d'instructions. Il est donc possible d'en donner une borne supérieure de manière analogue aux instructions.

Exemple : Soit l'instruction élémentaire i de multiplication effectuant le calcul $x := \text{mul}(y, z)$. Si une analyse numérique révèle que y vaut $[-1; 1]$ en entrée de i , et que z vaut $[100; 100]$, alors comme les configurations possibles sont $(-1, 100)$, $(0, 100)$, $(1, 100)$ les valeurs réelles de x en sortie de i seront -100 , 0 et 100 mais l'analyse nous fournira l'intervalle $[-100; 100]$, ajoutant donc un grand nombre de valeurs par sur-approximation. Nous avons donc un résultat d'analyse égal à 201 valeurs possibles, mais une borne $\text{sup}(i) = 3$. Il y a donc ici une perte de précision entraînant la création de 198 valeurs par sur-approximation.

De la même manière, nous voudrions détecter de la perte de précision sur des structures plus complexes. Cette réflexion nous amène à nous demander quelles sont les structures les plus susceptibles d'engendrer de la perte de précision.

2.5 Déterminer des structures particulières dans le CFG

Nous avons choisi de maximiser le nombre de structures pour l'analyse afin d'accélérer les étapes de raffinement. Il n'est pas toujours nécessaire d'avoir une analyse très précise sur une portion du programme pour analyser la suite. Par exemple, pour savoir si une indirection conditionnelle $\ll \text{jge @address} \gg$ est prise, il suffit de savoir que le calcul précédent a donné un résultat positif.

Nous avons déjà trois structures à notre disposition :

- L'instruction élémentaire.
- L'instruction assembleur.
- Le bloc de base.

Mais ces niveaux de détail ne sont pas suffisants pour nos besoins. En effet, ils ne prennent pas en considération la dynamique du programme, et ne peuvent servir au raffinement d'une boucle où d'une indirection. Nous allons donc introduire trois nouvelles structures qui serviront à notre analyse : la boucle, l'indirection et la séquence.

2.5.1 Structure de boucle

Les boucles sont des sources de perte de précision très importantes lors d'une analyse par interprétation abstraite. En effet, la valeur d'une variable x en entrée d'un bloc est définie par l'union des valeurs de x à la sortie de tous les blocs immédiatement précédents. Chaque tour de boucle ajoute donc de nouvelles valeurs

potentielles à l'ensemble d'entrée.

Il est donc important de pouvoir détecter une boucle, car ce sont ces structures que nous chercherons la plupart du temps à raffiner. Toutefois, dans le contexte de l'analyse d'un fichier binaire désassemblé, les boucles ne sont pas des structures toujours très *propres*. Par exemple, les programmes suivants sont des boucles :

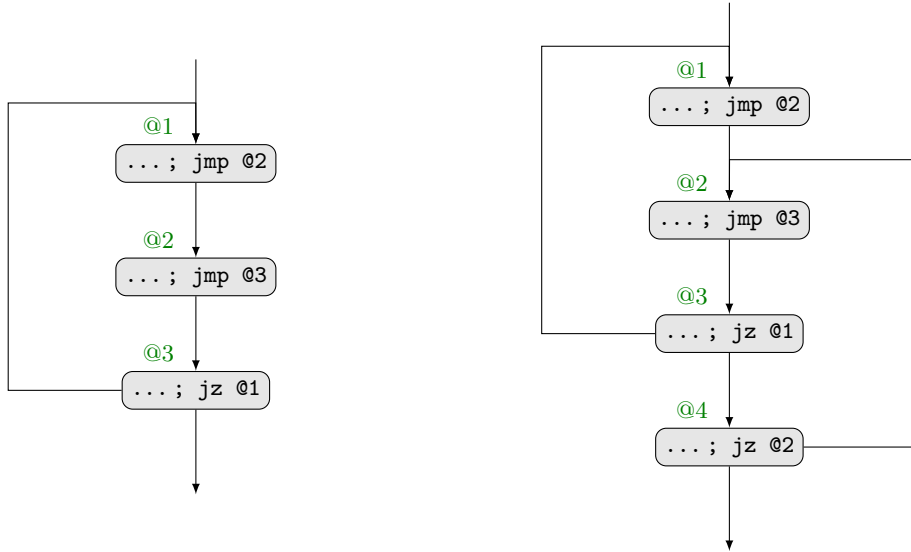


FIGURE 5 – Quelques boucles issues de programmes en langage assembleur Intel x86[1]

Nous ne pouvons donc pas simplement considérer un retour en arrière dans le graphe de flot de contrôle comme une boucle. La solution que nous avons choisie est issue de la théorie des graphes. Afin de déterminer quelles instructions appartiennent à une boucle, nous allons transformer notre CFG en graphe de composantes fortement connexes (CFC). De nombreux algorithmes pour obtenir ce graphe existent dans la littérature[17, 16].

Un graphe de composantes fortement connexes a une propriété très intéressante : c'est un graphe acyclique et orienté (DAG). Dans notre cas, cela signifie qu'il suffit donc de suivre le flot de contrôle pour faire converger notre programme. En effet, le graphe étant à présent acyclique, il n'y a pas possibilité de revenir en arrière, et donc le résultat d'un bloc b ne pourra jamais impacter celui d'un bloc b' s'il existe un chemin de b' à b au sein d'un tel graphe.

Il serait intéressant, une fois les CFC déterminées, de considérer celles-ci comme des programmes à part entière et d'y déterminer des sous-structures, par exemple en détruisant la structure de boucle (suppression d'une arête) de manière intelligente pour y retrouver des sous-boucles par une nouvelle recherche de CFC. Ce travail fera probablement l'objet d'un futur stage.

2.5.2 Structure d'indirection

Les indirections²⁰[9], correspondant par exemple²¹ aux instructions `<< if (cond) then ... else ... >>` dans des langages de plus haut niveau, sont aussi d'importantes sources de perte de précision. Comme pour les boucles, l'entrée d'un bloc est déterminée par l'union des sorties des blocs immédiatement précédents. Le problème classique est présenté dans le CFG suivant :

20. Nous utiliserons indifféremment les dénominations *indirection* et *branchement conditionnel* pour désigner ces structures, mais à proprement parler, une boucle est aussi une indirection. Nous avons préféré catégoriser ces dernières à part.

21. Ce n'est pas une généralité, juste un exemple ayant pour but de guider le lecteur dans la visualisation du concept.

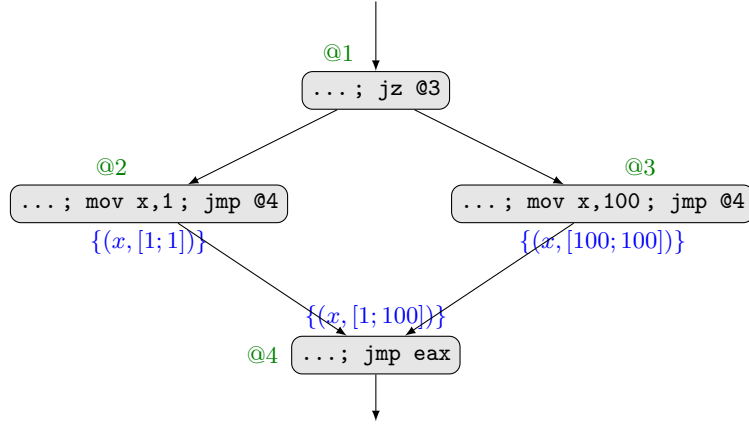


FIGURE 6 – Exemple de perte de précision engendrée par une indirection

Il est donc nécessaire de pouvoir raffiner l'analyse sur ce type de structures. Pour cela, nous disposons du CFG associé au programme, regroupé sous forme de CFC, que nous déplaçons à partir de v_0 (bloc d'adresse @1) :

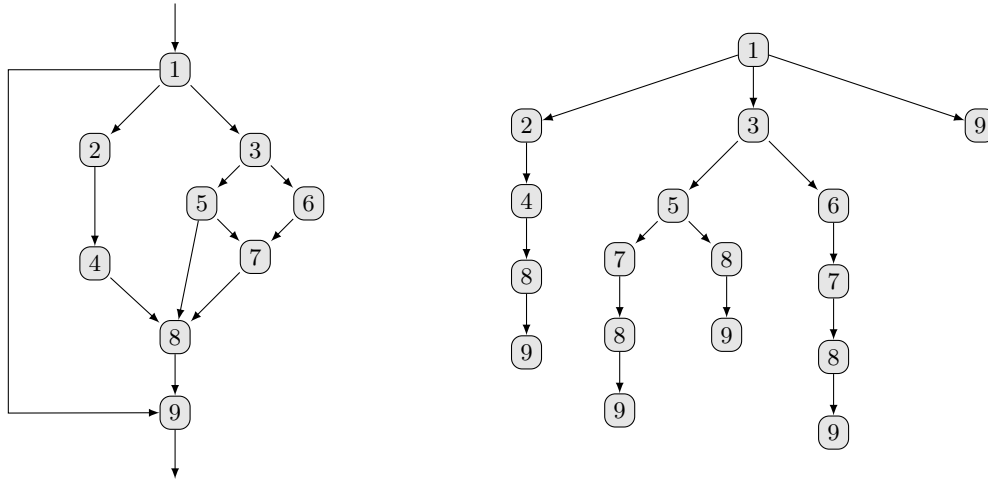


FIGURE 7 – Graphe sous forme de CFC et son dépliage à partir du premier bloc de base

Pour extraire les structures d'indirection intéressantes, nous procédons comme suit : étant donné un graphe déplié G , pour tout nœud n de G ayant plusieurs fils, nous cherchons le premier nœud n' tel que tout chemin partant de n passe par n' . Notons N_c l'ensemble des nœuds n_c tels qu'il existe un chemin $n \rightsquigarrow n_c \rightsquigarrow n'$. Le triplet (n, N, n') forme une indirection²².

Dans l'exemple donné en figure 7, nous avons les indirections $(1, \{2; 3; 4; 5; 6; 7; 8\}, 9)$ et $(3, \{5; 6; 7\}, 8)$.

Nous avons jugé ces structures comme étant les plus intéressantes car, de par leur construction, chacune ne peut avoir qu'une sortie. Il est toutefois possible de définir des structures plus élémentaires, en considérant par exemple le premier nœud n' commun à chaque couple de branches du graphe déplié à partir de n . Nous aurions alors avec cette méthode les indirections $(1, \{2; 3; 4; 5; 7\}, 8)$, $(1, \{2; 3; 4; 5\}, 8)$, $(1, \{2; 3; 4; 6; 7\}, 8)$, $(1, \{2; 4; 8\}, 9)$, $(1, \{3; 5; 8\}, 9)$, $(1, \{3; 5; 7; 8\}, 9)$, $(1, \{3; 6; 7; 8\}, 9)$, $(5, \{7\}, 8)$, $(3, \{5; 6\}, 7)$ et $(3, \{5; 6; 7\}, 8)$.

2.5.3 Séquence de structures

A ces deux types de blocs particuliers, nous ajoutons une structure plus élémentaire :

Séquence

Une séquence S est une suite de structures $(s_n)_n$ telle que si une structure s est exécutée, alors toutes les structures $s \in (s_n)_n$ le sont.

22. Dans le cas où il n'existe pas de nœud commun n' à l'indirection, nous ajoutons un nœud final n_f virtuel tel que le dernier bloc issu de n rejoigne n_f . Ce nœud virtuel devient donc n' et correspond aux sorties du programme.

Cette définition rappelle fortement celle d'un bloc de base (1.1.2), car le principe est le même mais à un niveau plus global. Il est important de noter que pour les indirections, cette définition ne s'applique qu'à la structure choisie, la seconde possibilité évoquée (couples de branches) permettant plusieurs sorties par structure.

Nous avons à présent les structures suivantes pour notre analyse :

- L'instruction élémentaire.
- L'instruction assembleur.
- Le bloc de base.
- La boucle²³.
- L'indirection.
- La séquence.

Majoration : Chacune des structures évoquées précédemment peut être vue comme une fonction déterministe qui, à une configuration des variables en entrée associe une nouvelle configuration de ces mêmes variables en sortie. Nous pouvons donc affirmer que pour toute structure, il existe au plus²⁴ autant de configurations des variables en entrée qu'en sortie.

2.6 Estimation par échantillonnage

Il nous faut à présent définir une borne minimale sur le cardinal des sorties. Pour cela, nous allons procéder de manière dynamique, en utilisant des techniques de *fuzzing*²⁵. Nous analysons le code constituant la structure pour trouver des relations entre les variables utilisées (par exemple, en utilisant des méthodes statistiques[20]). Si par exemple une structure s utilise les variables x et y dans la relation $x = 2y$, alors nous pouvons définir un treillis relationnel entre x et y ²⁶[5, 6].

Nous pouvons alors choisir un nombre $k \leq \text{sup}(s)$ de n -uplets, où n est le nombre de variables manipulées par la structure s . A présent, évaluons le code de s avec les valeurs choisies en entrée²⁷. Pour chaque sortie j de s , nous disposons alors de bornes $\text{inf}(s_j)$ correspondant au nombre de configurations distinctes détectées sur la sortie j pour chaque configuration d'entrée.

Nous disposons alors de bornes inférieures et supérieures pour les sorties d'une structure s . Il serait à présent intéressant de définir une heuristique pour déterminer dans quelle partie de l'intervalle $[\text{inf}(s_j); \text{sup}(s)]$ se trouve le nombre réel de configurations en sortie j de s ²⁸. Pour cela, nous pouvons définir sur chaque sortie j de s une valeur $\overline{s_j} = \frac{\text{inf}(s_j)}{k} * n$, où k est le nombre d'échantillons utilisés pour le *fuzzing* parmi n possibles.

Avec cette relation, si la structure analysée est une injection²⁹, nous aurons un faible nombre de résultats pour $\text{inf}(s_j)$, et en conservant le rapport $\frac{\text{inf}(s_j)}{k}$ sur le nombre d'entrées possibles, nous obtenons une valeur $\overline{s_j} \in [\text{inf}(s_j); \text{sup}(s)]$ qui sera potentiellement proche de $\text{inf}(s_j)$.

Nous avons donc défini trois indicateurs pour juger de la précision d'une interprétation abstraite. Il reste donc à définir une heuristique pour déterminer, étant donné le résultat d'une analyse sur une structure s , si la perte de précision est importante et mérite un raffinement. Pour cela, nous pouvons prendre en compte un certain nombre de données comme par exemple :

- Le type de structure : il sera probablement plus intéressant de gagner en précision sur l'analyse d'une boucle ou d'une indirection que sur celle d'un bloc de base qui engendre moins de perte de précision.
- L'emplacement au sein du programme : une perte de précision ayant lieu au tout début de l'analyse d'un programme se répercutera dans la suite, et pourra entraîner l'analyse de mauvaises branches d'une indirection.
- L'appartenance à une trace d'exécution choisie : si le but de l'analyse est de raffiner la destination d'un saut à destination variable particulier, il n'est alors pas utile de gagner en précision sur les branches ne

23. Ainsi qu'éventuellement les sous-structures y étant incluses.

24. Il est possible que deux configurations d'entrée engendrent une même configuration de sortie. Dans ce cas, nous ne conservons pas le doublon.

25. Technique consistant à tester un nombre choisi de valeurs prises au hasard (ou non) dans un ensemble.

26. Un treillis relationnel est un treillis à plusieurs dimensions. Les octogones[13], les zones ou encore les polyèdres convexes sont des exemples de tels treillis.

27. Ici aussi, il conviendra de placer une *garde* pour assurer la terminaison.

28. Dans le cas d'un faible nombre de configurations possibles en entrée, une évaluation exhaustive est plus intéressante.

29. Au sens mathématique du terme.

menant pas à ce saut.

- L'écart entre le cardinal des valeurs possibles d'une variable x et $\overline{s_j}$: cela permet de détecter par exemple une injection.
- Le dépassement de $\sup(s)$ par le nombre de configurations détectées en sortie d'une structure s : cela dénote une perte de précision, et indique potentiellement une nécessité de changer de mode de représentation pour les variables.

Une fois qu'un problème est détecté via une heuristique choisie³⁰, il reste à le résoudre pour pouvoir continuer l'analyse. Pour cela, nous distinguons deux cas :

- Le nombre de n -uplets en entrée de la structure s est fini et d'une taille acceptable³¹ : dans ce cas, nous effectuons une évaluation de s au sein d'une machine virtuelle pour chaque configuration possible en entrée de s . Si l'évaluation termine bien, une nouvelle étape d'interprétation abstraite peut avoir lieu. Dans le cas contraire (utilisation d'une garde), nous fournissons à l'analyste les traces d'exécution parcourues afin qu'il intervienne manuellement dans le processus.
- Le nombre de n -uplets en entrée de la structure s est infini ou d'une taille inacceptable : nous demandons à l'analyste de sélectionner un nombre fini de valeurs à tester, qu'il juge caractéristiques des valeurs en entrée.

Dans les cas où un appel à l'analyste s'avère nécessaire, il est important de noter que nous perdons la complétude de l'analyse. Ce choix n'est pas toujours approprié, mais peut suffire à prouver un certain nombre de propriétés sur le programme, comme la possibilité de prendre une indirection du type `<< jz @address >>` (il suffit d'une valeur activant la condition de saut).

30. Les idées présentées ci-dessus sont des pistes de réflexion. La mise en place d'une heuristique efficace est une continuation possible de nos travaux, et nécessitera une implémentation fonctionnelle pour validation.

31. La définition du terme *acceptable* est un paramètre déterminé par l'analyste.

Conclusion

Le couplage des techniques d'interprétation abstraite et d'évaluation de code permet donc la détection de perte de précision induite par une analyse. Cela permet de mettre en avant des zones du code où un raffinement de l'analyse serait intéressant afin de caractériser les registres contenant les destinations des sauts du type « `jmp eax` ». Nous avons défini un certain nombre de structures permettant une détection à plusieurs niveaux, et avons proposé une technique d'évaluation de la perte de précision d'une analyse par interprétation abstraite, basée sur la détection de bornes sur les cardinaux des ensembles de valeurs.

Un début d'implémentation en Java³² a été réalisé, pour mesurer les performances en terme de précision de notre technique par rapport à une analyse par interprétation abstraite standard. Ce logiciel ne réalise pour l'instant que l'interprétation abstraite et l'émulation d'un code de manière séparée, mais n'effectue pas encore le mélange des deux techniques. De plus, seuls des treillis non-relationnels sont implémentés. La continuation de ce logiciel pourra faire l'objet d'un futur stage.

Un autre axe de travail intéressant serait la recherche de sous-structures au sein des composantes fortement connexes. Cela permettrait de bénéficier de la possibilité de raffiner certaines parties du code actuellement *camouflées* dans des boucles, et de cibler plus précisément les sources des pertes de précision.

En conclusion, le mélange de techniques d'interprétation abstraite et d'évaluation de code pour détecter les pertes de précision est un axe de recherche qui semble très prometteur, mais il reste encore une grande quantité de travail à effectuer, notamment sur l'implémentation, puis sur la définition d'une heuristique de détection de problèmes efficace, ainsi que sur la résolution automatique de ceux-ci.

32. Ce langage a été utilisé pour sa facilité d'appréhension, et donc pour pouvoir permettre une continuation future.

Références

- [1] Intel 64 and IA-32 Architectures Software Developer's Manual, 2012.
- [2] Jean-Marie Borello. *Etude du métamorphisme viral : modélisation, conception et détection*. PhD thesis, Université de Rennes 1 & Supélec, 2011.
- [3] Patrick Cousot & Radha Cousot. Abstract interpretation : a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference Record of the Sixth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1977.
- [4] Fabrice Desclaux. Miasm. In *SSTIC (Symposium sur la Sécurité des Technologies de l'Information et de la Communication)*, 2012.
- [5] Jérôme Feret. Numerical abstract domains for digital filters. 2005.
- [6] Jérôme Feret. Static analysis of digital filters. In *European Symposium on Programming (ESOP'04)*, 2006.
- [7] Eric Filiol. *Les virus informatiques : théorie, pratique et applications*. Springer, 2009.
- [8] Eric Filiol. Pwn2kill challenge debrief. Technical report, ESIEA, CVO, 2010.
- [9] Roberto Giacobazzi. Gaming obscurity with interpreters : a journey on code obfuscation by formal methods. 2012.
- [10] Flemming Nielson & Hanne Riis Nielson & Chris Hankin. *Principles of Program Analysis*. Springer, 2010.
- [11] Johannes Kinder & Dimitri Kravchenko. Alternating control flow reconstruction. In *VMCAI Proceedings of the 13th international conference on Verification, Model Checking, and Abstract Interpretation*, 2012.
- [12] Felix Leder & Bastian Steinbock & Peter Martini. Classification and detection of metamorphic malware using value set analysis. *IEEE conference publication*, 2009.
- [13] Antoine Miné. The octagon abstract domain. In *In AST 2001 in WCRE 2001, IEEE*, 2001.
- [14] Thomas Dullien & Sebastian Porst. Reil : A platform-independent intermediate representation of disassembled code for static code analysis. In *Proceeding of CanSecWest*, 2009.
- [15] Gogul Balakrishnan & Thomas Reps. Wysiwyx : What you see is not what you execute. *ACM Transactions on Programming Languages and Systems*, 2010.
- [16] Micha Sharir. A strong connectivity algorithm and its applications to data flow analysis. *Computers and Mathematics with Applications* 7, 1981.
- [17] Robert E. Tarjan. Depth first search and linear graph algorithms. *SIAM Journal of Computing*, 1972.
- [18] Sébastien Bardin & Philippe Herrmann & Jérôme Leroux & Olivier Ly & Renaud Tabary & Aymeric Vincent. The bincoa framework for binary code analysis. In *Proc. of the 23rd International Conference on Computer Aided Verification (CAV'11)*, 2011.
- [19] Sébastien Bardin & Philippe Herrmann & Franck Védrine. Refinement-based cfg reconstruction from unstructured programs. In *VMCAI Proceedings of the 12th international conference on Verification, Model Checking, and Abstract Interpretation*, 2011.
- [20] Bert G. Wachsmuth. Correlation between variables, consulted 08/16/2012. <http://pirate.shu.edu/~wachsmut/Teaching/MATH1101/Relations/correlation.html>.

Annexes

A Langage intermédiaire

Le but d'un langage intermédiaire est d'expliciter l'intégralité des effets de bord des instructions du langage assembleur, ainsi que de se libérer des spécificités du langage choisi. Cela permet l'utilisation d'un algorithme d'interprétation abstraite plus générique car indépendant de la plateforme d'exécution du programme analysé.

Une première transformation à effectuer est le passage d'un langage CISC à un langage RISC. En effet, des instructions telles que `<< loop @address >>` sont de véritables algorithmes. Cette instruction composée peut par exemple être transformée en la séquence d'instructions `<< dec cx ; jz @address >>` utilisant des instructions plus élémentaires.

La seconde transformation consiste à expliciter les effets de bord des instructions en langage RISC. Pour cela, nous définissons le langage intermédiaire constitué des instructions suivantes :

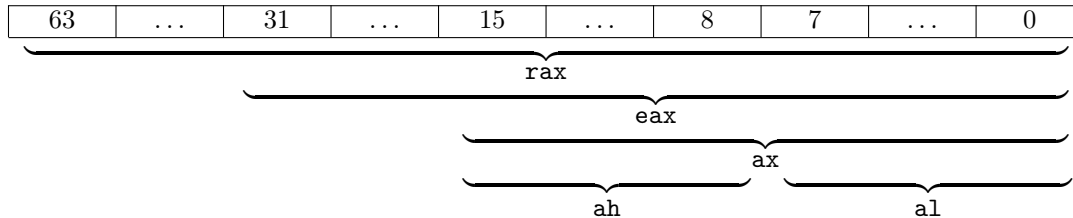
- $var_d := \mathbf{load}(\mathbf{mem})$ ³³ : charge le contenu d'une mémoire dans une variable.
- $\mathbf{store}(var_s, \mathbf{mem})$: charge le contenu d'une variable dans une mémoire.
- $var_d := \mathbf{size}(\mathbf{mem})$: fonction renvoyant la taille en bits de la mémoire passée en paramètre.
- L'ensemble des opérations arithmétiques et logiques standard ne manipulant que des variables ($var_d := \mathbf{add}(var_{s_1}, var_{s_2})$; $var_d := \mathbf{not}(var_s)$; ...).

De plus, nous nous autorisons l'utilisation de *macros*³⁴, que nous noterons en lettres capitales. Par exemple, $\mathbf{FIT}(var_d, var_s, size)$ est une macro qui, étant données une variable var_s et une taille $size$, effectue la troncature de var_s sur $size$ bits et stocke ce résultat dans var_d . Cette macro sera transformée en la séquence d'instructions élémentaires `<< mask := pow(2, size) ; mask := sub(mask, 1) ; var_d := and(var_s, mask) >>`.

Ainsi, avec ce langage intermédiaire et quelques macros non présentées ici, l'instruction en langage assembleur `<< add eax, ebx >>` est décomposée comme suit :

- | | |
|--|---|
| <ol style="list-style-type: none"> 1. INCREMENT(<i>eip</i>³⁵) 2. $eax := \mathbf{load}(eax)$ 3. $ebx := \mathbf{load}(ebx)$ 4. $size := \mathbf{size}(eax)$ 5. $res := \mathbf{add}(eax, ebx)$ 6. UPDATE_ZF($res, size$) 7. UPDATE_CF($res, size$) | <ol style="list-style-type: none"> 8. UPDATE_PF($res, size$) 9. UPDATE_OF($res, size$) 10. UPDATE_SF($res, size$) 11. UPDATE_AF($res, size$) 12. FIT($res_{size}, res, size$) 13. $\mathbf{store}(res_{size}, eax)$ |
|--|---|

Une étape de plus, non précisée ici, s'avère nécessaire. Si un registre comme **eax** est modifié, il faut aussi déterminer les nouvelles valeurs des registres **rax**, **ax**, **al** et **ah**. Cela nécessite une configuration propre au langage assembleur, permettant la prise en compte d'une disposition superposée de registres :



Une autre solution pour prendre en compte cette spécificité est de considérer un treillis par bit et non par élément mémoire. Le registre englobant **rax** est alors vu comme un vecteur de bits.

33. Nous utiliserons les conventions suivantes : les variables locales, purement virtuelles, seront notées en italique, les mots-clés du langage intermédiaire seront en gras, et les mémoires élémentaires auront une police à chasse fixe.

34. Chaîne de caractère remplacée par sa valeur à l'emplacement où elle se trouve.

35. Nous remarquons ici l'utilisation de registres comme **eip** ou encore **eax**. Ces notations ont pour but d'aiguiller le lecteur, mais seront transformées en jetons dans une implémentation réelle pour faire abstraction de la syntaxe x86[1].

B Algorithme d'interprétation abstraite

La fonction principale, *abstractInterpretation*, prend en entrée un programme à analyser, une fonction permettant de vérifier un certain nombre de spécifications, une fonction d'abstraction α et une fonction de concrétisation γ . L'algorithme commence par créer le CFG associé au programme, ainsi que la fonction f qui permettra la convergence du système. Ensuite, il applique l'ensemble des équations du système séquentiellement³⁶ pour faire converger un système initialisé à $\perp$ ³⁷.

Une fois le système convergé, la fonction annexe *specsFunction* est appelée pour vérifier les propriétés désirées. Un exemple d'une telle fonction est donné par l'algorithme 3, pour le programme donné en figure 1. La fonction récupère la composante du système contenant les informations recherchées, ici $exit_{i_2 \rightarrow end}$, et y lit la valeur de la variable x (dont le domaine abstrait est le treillis des signes) permettant de donner une réponse à la spécification $\forall n, fact(n) > 0$.

Algorithm 1 *abstractInterpretation(program, specsFunction, α, γ)*

```

cfg := createCFG(program)38
f :=  $\alpha \circ createSystem(program)$ 39  $\circ \gamma$ 
values1 :=  $\perp$ 
values2 := applySystem(f, values1, cfg)
while values1  $\neq$  values2 do
    tmp := values2
    values2 := applySystem(f, values1, cfg)
    values1 := tmp
end while
return specsFunction(values1)

```

Algorithm 2 *applySystem(transitionFunction, values, cfg)*

```

equations := sortEquations(transitionFunction, cfg)38
for each oneEquation in equations do
    values := applyEquation(oneEquation, values)40
end for
return values

```

Algorithm 3 *specsFunction(values)*

```

resultComponent := getComponent(values, exiti2 → end)
resultVariable := getVariable(resultComponent, 'x')
return resultVariable = '+'

```

Par exemple, pour le programme en figure 1, si chaque variable a pour domaine abstrait le treillis des signes $L^\# = (\{-, 0, +\}, \sqsubseteq^\#, \sqcup^\#, \cap^\#, \emptyset^\#, -0+)\text{,}$ le système de base et son point fixe sont donnés par ces systèmes :

$$\vec{P}_0 = \begin{cases} entry_{i_1} = \perp \\ exit_{i_1 \rightarrow i_2} = \perp \\ entry_{i_2} = \perp \\ exit_{i_2 \rightarrow i_3} = \perp \\ exit_{i_2 \rightarrow end} = \perp \\ entry_{i_3} = \perp \\ exit_{i_3 \rightarrow i_4} = \perp \\ entry_{i_4} = \perp \\ exit_{i_4 \rightarrow i_2} = \perp \end{cases} \quad \vec{P}_{lfp} = \begin{cases} entry_{i_1} = \{(n, \top)\} \\ exit_{i_1 \rightarrow i_2} = \{(n, \top), (x, +)\} \\ entry_{i_2} = \{(n, \top), (x, +)\} \\ exit_{i_2 \rightarrow i_3} = \{(n, +), (x, +)\} \\ exit_{i_2 \rightarrow end} = \{(n, \top), (x, +)\} \\ entry_{i_3} = \{(n, +), (x, +)\} \\ exit_{i_3 \rightarrow i_4} = \{(n, +), (x, +)\} \\ entry_{i_4} = \{(n, +), (x, +)\} \\ exit_{i_4 \rightarrow i_2} = \{(n, 0+), (x, +)\} \end{cases}$$

36. Il est possible de montrer que l'application séquentielle des équations associées au programme revient au même que l'application simultanée de celles-ci. L'utilisation du CFG dans cet algorithme est donc une simple heuristique permettant de choisir de manière plus optimisée l'ordre des équations à appliquer.

37. $\vec{\perp}$ dénote un vecteur à n composantes si f est constituée de n équations, dans lequel chaque variable est initialisée au plus petit élément du treillis sur lequel elle est définie.

38. Méthode extérieure au contexte du rapport, dont le détail ne sera pas donné ici.

39. Le fonctionnement de cette méthode est décrit en section 1.2.1.

40. Application de f pour une des équations de $\vec{P}$, selon les systèmes décrits en 3 et 1.2.3.

Il est important de noter qu'ayant mis de côté l'opérateur ∇ de *widening* dans le cadre de ce rapport, l'algorithme présenté ici peut ne pas terminer. L'introduction d'une telle notion aurait compliqué l'algorithme, et nous avons préféré considérer cette annexe comme un outil d'aide à la compréhension du rapport plus que comme une preuve formelle.

C Chaîne d'analyse

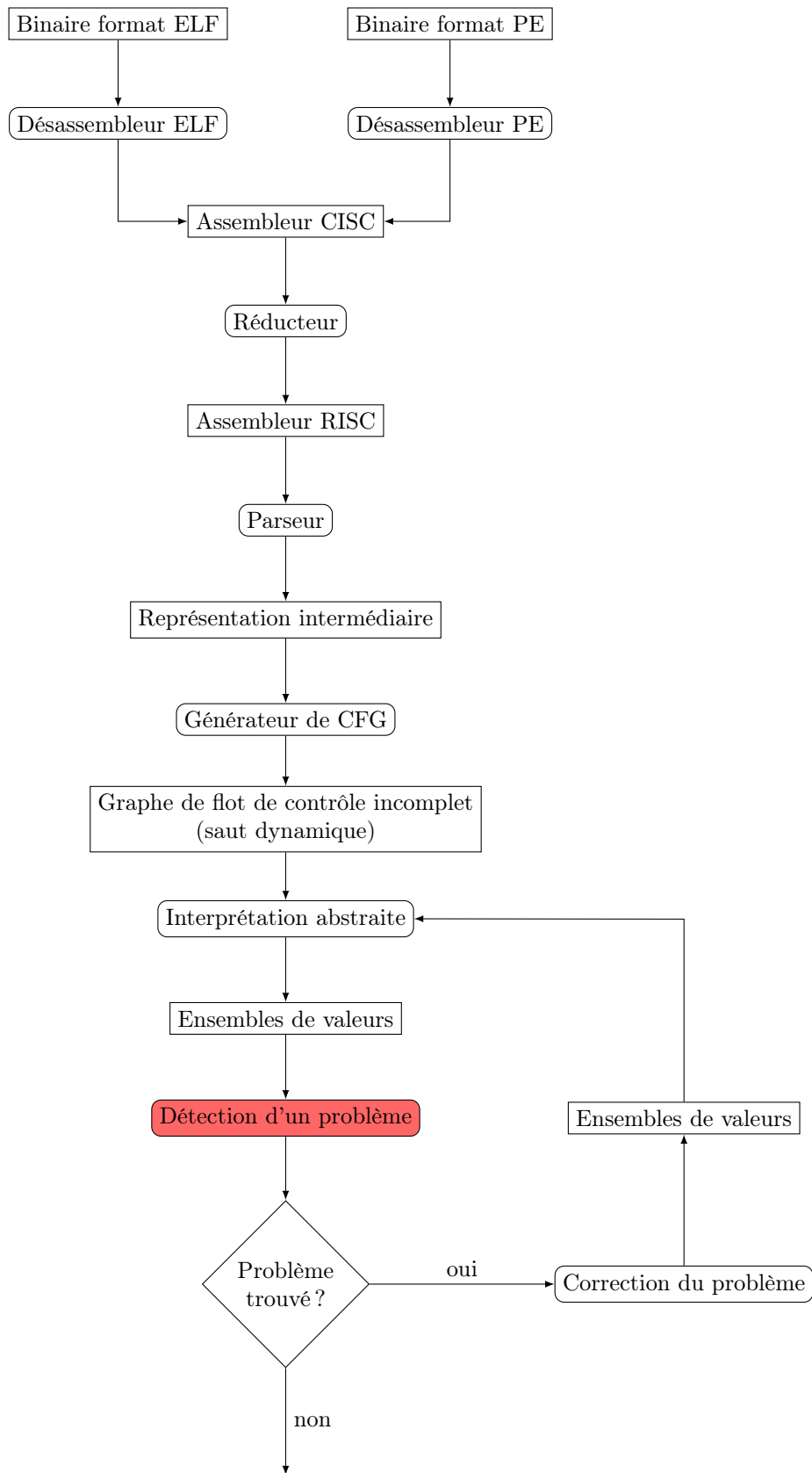


FIGURE 8 – Chaîne d'analyse complète